

TP - Multidimensional Indexing and Coalesced Memory Access

Oguz Kaya

oguz.kaya@universite-paris-saclay.fr

Part 1

The exercise: one saxpy, eighteen mappings, two layouts

In this lab you will write *one* computation many times: the saxpy operation $Y \leftarrow aX + Y$ on two $N \times N$ single-precision matrices X and Y ($N = 256$). What changes from one version to the next is only *which thread computes which element*, following the cases (a)–(d) and their alternatives seen in the lecture, and how the matrix is stored in memory. The goal is to decide, for every version, whether the memory accesses of a warp are **coalesced**.

The matrices are allocated as *one-dimensional* arrays of $N \cdot N$ floats (`malloc` on the CPU, `cudaMalloc` on the GPU). How the element (i, j) is placed in that array is a convention that you choose, and every kernel exists in two versions, one per convention:

- **row-major** (C convention): element (i, j) is at $A[i * N + j]$; kernel suffix `_rowmajor`;
- **column-major** (Fortran/BLAS convention): element (i, j) is at $A[i + j * N]$; kernel suffix `_colmajor`.

The two versions of a kernel differ in exactly one line, the computation of the position `idx` of element (i, j) ; write everything by hand in each kernel, without helper functions.

You will work in the provided skeleton code `saxpy2d.cu`. It contains the eighteen kernel pairs with a comment block above each one describing the mapping to implement, and a driver that launches every kernel in both layouts with the grid and block dimensions given in the function `config()`, checks the result $Y = aX + Y$ (column `result`), and verifies the mapping as explained below. Compile and run it with

```
nvcc -O2 -arch=native saxpy2d.cu -o saxpy2d ./saxpy2d
```

How your kernels are checked

A kernel that produces the right values of Y may still implement the wrong mapping (interchange `blockIdx.x` and `blockIdx.y`, or `threadIdx.x` and `threadIdx.y`, and the result is unchanged). The driver therefore runs every kernel in *trace mode*: right after each assignment to `Y[idx]`, the macro `CHECK_INDEX()` records the four indices `blockIdx.x`, `blockIdx.y`, `threadIdx.x`, `threadIdx.y` of the thread that updated element `idx`. The CPU then checks that every element was touched exactly once (column `covered`), that it was touched by the block and thread the mapping prescribes (column `mapping`), and counts the 32-byte memory sectors touched by the 32 threads of a warp at one instruction (column `sectors/warp`): 4 sectors for 32 floats means the access is coalesced, 32 sectors means that every thread fetches its own sector and 7/8 of the transferred bytes are wasted. For this to work, **every** assignment to `Y[idx]` in your kernels must be followed by `CHECK_INDEX()`; and the position must be stored in a variable named `idx`, as in the given kernels. The function `expected()` of the driver is the formal specification of every mapping; you may read it, but do not modify the driver.

Kernels 0 (the plain 1D saxpy over the N^2 elements, our reference speed) and 1 (one thread per row) are given as worked examples, including their verdicts. For each of the other kernels, marked `TODD`, you have to:

1. write the body of the `_rowmajor` and `_colmajor` kernels following the mapping described in the comment block above them;
2. in the comment block, describe what the 32 threads of a warp touch in memory at one instruction, for both layouts, and conclude whether the access is coalesced or not;
3. write the same verdict ("yes"/"no") in the table `kernels[]`: the program compares it with the number of sectors it measures and flags a disagreement.

Remember that the 32 threads of a warp are 32 consecutive values of `threadIdx.x` (with the same `threadIdx.y`), that the elements of a row are contiguous in the row-major layout and the elements of a column in the column-major one, and that the numbering of the blocks has no effect on what a warp touches.

Part 2

One thread, one row or one column*Ex. 1*

- a) Read the two given kernels `saxpy_flat` and `saxpy_row_per_thread_*`, compile, run, and check that you understand every column of the output for them.
- b) Write `saxpy_col_per_thread_rowmajor` and `saxpy_col_per_thread_colmajor` (kernel 2): one thread per column, looping over the rows. Comments and verdicts as explained above.

Part 3

1D blocks along a row or a column: cases (a) and (b)

Each block handles `BLOCK_SIZE = 128` consecutive elements of one row (case (a), threads row aligned) or of one column (case (b), threads column aligned). The block coordinates (b_i, b_j) are obtained either by arithmetic on a 1D grid, or directly from a 2D grid.

Ex. 2

- a) `saxpy_1d_a_*` (kernel 3): 1D grid of $N \cdot \lceil N/128 \rceil$ blocks numbered row by row; $b_i = \text{blockIdx.x} / \text{blocksPerRow}$, $b_j = \text{blockIdx.x} \% \text{blocksPerRow}$.
- b) `saxpy_1d_b_*` (kernel 4): the same along the columns, blocks numbered column by column.
- c) `saxpy_2d_a_*` (kernel 5): case (a) again, but with a 2D grid $N \times \lceil N/128 \rceil$ so that `blockIdx.x` and `blockIdx.y` give b_i and b_j without any arithmetic.
- d) `saxpy_2d_b_*` (kernel 6): case (b) with a 2D grid $\lceil N/128 \rceil \times N$.
- e) Compare the verdicts of kernels 3 and 5, then 4 and 6. What does the way of numbering the blocks change?

Part 4

Tile blocks with a 1D grid: cases (c), (d) and their alternatives

Each block now handles a 32×32 tile of the matrix (`SIDE = 32`, $32 \cdot 32 = 1024$ threads). The grid is 1D with $\lceil N/32 \rceil^2$ blocks, and both the block coordinates (b_i, b_j) and the thread coordinates (t_i, t_j) inside the tile are computed by division and modulus.

Ex. 3

- a) `saxpy_1d_c_*` (kernel 7): tiles numbered row by row, threads *column aligned*: $t_i = \text{threadIdx.x} \% \text{SIDE}$, $t_j = \text{threadIdx.x} / \text{SIDE}$, so that 32 consecutive threads go down one column of the tile.
- b) `saxpy_1d_d_*` (kernel 8): same tiles, threads *row aligned*: $t_i = \text{threadIdx.x} / \text{SIDE}$, $t_j = \text{threadIdx.x} \% \text{SIDE}$.
- c) `saxpy_1d_c_alt_*` and `saxpy_1d_d_alt_*` (kernels 9 and 10): the same two kernels with the tiles numbered column by column ($b_j = \text{blockIdx.x} / \text{tilesPerCol}$, $b_i = \text{blockIdx.x} \% \text{tilesPerCol}$).
- d) Before running, predict the verdicts of kernels 9 and 10 from those of kernels 7 and 8, then check.

Part 5

Tile blocks with 2D grids and 2D blocks

The same four mappings again, this time with `dim3` grids and blocks so that no division or modulus is needed: the grid is $\lceil N/32 \rceil \times \lceil N/32 \rceil$ and the blocks are 32×32 .

Ex. 4

- a) `saxpy_2d_c_*` (kernel 11): $t_i = \text{threadIdx.x}$, $t_j = \text{threadIdx.y}$, $b_i = \text{blockIdx.x}$, $b_j = \text{blockIdx.y}$. Be careful to multiply each block coordinate by the extent of the thread index it is combined with (`blockDim.x` or `blockDim.y`).
- b) `saxpy_2d_d_*` (kernel 12): threads row aligned, $t_i = \text{threadIdx.y}$, $t_j = \text{threadIdx.x}$.
- c) `saxpy_2d_c_alt_*` and `saxpy_2d_d_alt_*` (kernels 13 and 14): the grid is transposed, $b_i = \text{blockIdx.y}$, $b_j = \text{blockIdx.x}$.
- d) Which of the kernels 7–14 have exactly the same verdicts? Explain in one sentence what determines the verdict.

Part 6

Several elements per thread

Finally, each thread of kernels 11–14 computes $K \times K$ elements instead of one, with $K = \text{KK} = 4$ passed as a kernel argument. A block then covers a $32K \times 32K$ tile of the matrix, and the grid is $\lceil N/32K \rceil \times \lceil N/32K \rceil$. The $K \times K$ elements of a thread must be *interleaved* with those of the other threads of the block, so that thread (t_i, t_j) computes the elements

$$i = b_i \cdot 32K + t_i + 32 d_i, \quad j = b_j \cdot 32K + t_j + 32 d_j, \quad 0 \leq d_i, d_j < K.$$

Ex. 5

- a) Write `saxpy_2d_c_kxk_*`, `saxpy_2d_d_kxk_*`, `saxpy_2d_c_alt_kxk_*` and `saxpy_2d_d_alt_kxk_*` (kernels 15–18). Each thread performs K^2 assignments to `Y[idx]`, each followed by `CHECK_INDEX()`.
- b) Why are the verdicts of kernels 15–18 the same as those of kernels 11–14? What would the 32 threads of a warp touch if, instead, each thread computed a *contiguous* $K \times K$ patch ($i = (b_i \cdot 32 + t_i)K + d_i$, and likewise for j)? How many sectors per warp instruction would that be?